WP bloggar

Server-Side WordPress Activity Logging and File Integrity Monitoring

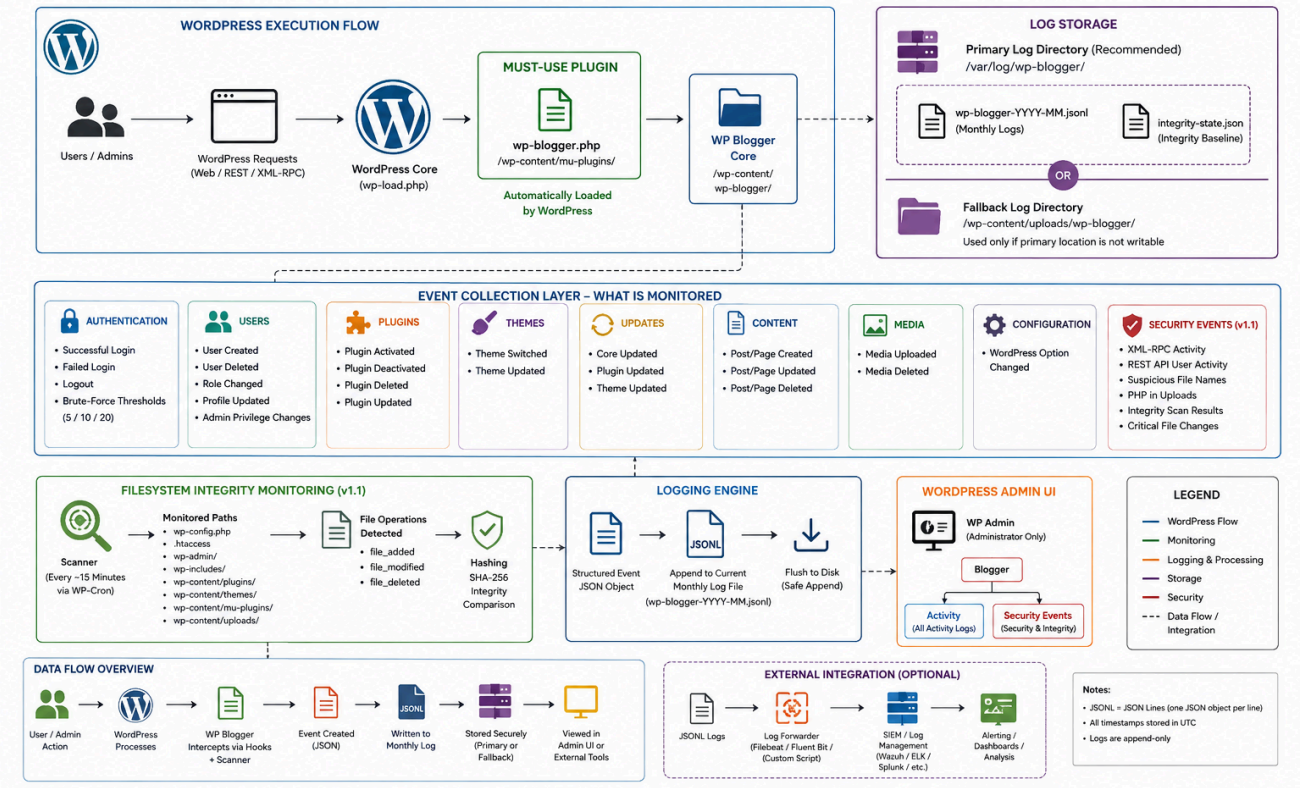
Version: 1.1.0
Type: WordPress Must-Use Plugin
Log Format: JSON Lines (JSONL)
Log Rotation: Monthly
Primary Log Location: `/var/log/wp-bloggar/`

Niranjan Meegammanma|
Shilpa Sayura Foundation

1. Overview

WP bloggar is a lightweight server-side security activity logger for WordPress.

It is designed to provide administrators and security teams with visibility into important WordPress activities including authentication events, user administration, plugin and theme changes, content modifications, configuration changes and filesystem integrity events. It operates as a **WordPress Must-Use (MU) plugin** rather than a conventional WordPress plugin.



WP: Blogar provides several operational advantages:

- automatically loaded by WordPress;
- no normal plugin activation is required;
- cannot normally be deactivated through the standard Plugins interface;
- logging code can be separated from the standard plugin directory;
- logs can be stored outside the public web root;
- supports filesystem integrity monitoring;
- provides structured JSONL logs suitable for later SIEM integration.

WP bloggar is intended as an additional auditing and detection layer. It does not replace server hardening, endpoint protection, backups, vulnerability management, Web Application Firewalls, SIEM or professional incident-response controls.

2. Objectives

WP bloggar was designed around five primary objectives:

1. **Activity Visibility**
   Record security-relevant actions occurring inside WordPress.
2. **Administrative Accountability**
   Record actions involving users, administrators, plugins, themes and configuration.
3. **Filesystem Integrity Monitoring**
   Detect unexpected creation, modification or deletion of important WordPress files.
4. **Web-Shell Detection Support**
   Identify potentially suspicious PHP files, particularly executable files appearing in locations such as the uploads directory.
5. **Incident Investigation**
   Maintain structured server-side records that can assist forensic analysis following a suspected compromise.

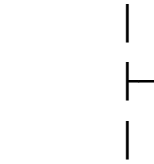
3. Architecture

WP bloggar uses a small MU-plugin bootstrap to load the main logging system.

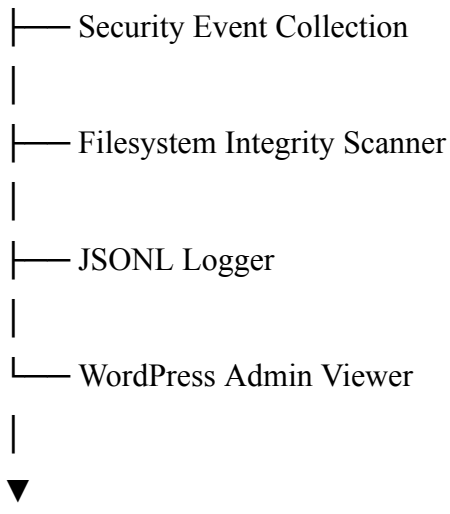
WordPress

wp-content/mu-plugins/wp-bloggar.php

wp-content/wp-bloggar/



├── Activity Event Collection



/var/log/wp-bloggar/

The MU bootstrap is automatically loaded during WordPress execution.

The primary application code is maintained separately under:

/wp-content/wp-bloggar/

This separation avoids placing the complete logger inside the conventional WordPress plugin directory.

4. Installation Structure

Recommended installation:

```
wordpress/
│
├── wp-admin/
├── wp-includes/
├── wp-content/
│   │
│   ├── mu-plugins/
│   │   └── wp-bloggar.php
│   │
│   └── wp-bloggar/
│       ├── admin/
│       ├── includes/
```

```
│       └── other application components
│
└── wp-config.php
```

The primary logging directory is:

/var/log/wp-bloggar/

If the primary directory cannot be used, the configured fallback location is:

/wp-content/uploads/wp-bloggar/

Using the primary location is strongly recommended.

5. Why a Must-Use Plugin?

Normal WordPress plugins generally reside under:

/wp-content/plugins/

and can normally be activated and deactivated through WordPress administration.

WP bloggar instead uses:

/wp-content/mu-plugins/

WordPress automatically loads PHP files located directly in this directory.

This means WP bloggar does not require:

Plugins → Activate

after deployment.

An administrator should be able to confirm its presence through:

Plugins → Must-Use

depending on the WordPress installation and administrative permissions.

Security limitation

MU-plugin deployment should not be interpreted as making the software invisible or tamper-proof.

A user or attacker with sufficient server filesystem privileges can discover, modify or remove MU-plugin files.

Security therefore depends primarily on:

- Linux filesystem permissions;
- ownership controls;
- restricted SSH access;
- secure WordPress administration;
- log protection;
- integrity monitoring;
- server hardening.

Obscurity is not treated as a primary security control.

6. Log Storage

Primary Location

/var/log/wp-bloggar/

This location is preferred because it exists outside the WordPress web root.

Example:

/var/log/wp-bloggar/

├── wp-bloggar-2026-08.jsonl

├── wp-bloggar-2026-09.jsonl

└── integrity-state.json

Fallback Location

If the primary location cannot be written, WP bloggar can use:

/wp-content/uploads/wp-bloggar/

Because this location is inside the web application tree, server configuration should prevent direct HTTP access to log files.

The fallback should be regarded as a resilience mechanism rather than the preferred production configuration.

7. Linux Log Directory Setup

Example Apache configuration using the common `www-data` service account:

sudo mkdir -p /var/log/wp-bloggar

sudo chown root:www-data /var/log/wp-bloggar

sudo chmod 770 /var/log/wp-bloggar

Verify:

ls -ld /var/log/wp-bloggar

The exact web-server account varies between operating systems and hosting environments.

Do not blindly change permissions to `777`.

8. Monthly Log Rotation

WP bloggar uses monthly JSONL log files.

Naming convention:

wp-bloggar-YYYY-MM.jsonl

Examples:

wp-bloggar-2026-08.jsonl

wp-bloggar-2026-09.jsonl

wp-bloggar-2026-10.jsonl

A new monthly file is therefore used as the calendar month changes.

This simplifies:

- archival;
- forensic collection;
- searching;
- SIEM ingestion;
- retention management;
- backup management.

9. JSON Lines Format

WP bloggar uses **JSON Lines (JSONL)**.

Each line represents one independent security or activity event.

Conceptual example:

```
{
  "timestamp": "2026-08-11T16:21:42Z",
  "event_id": "login_success",
  "severity": "info",
  "user_id": 1,
  "username": "administrator",
  "source_ip": "192.0.2.10",
  "request_uri": "/wp-admin/",
  "details": {
    "message": "User successfully authenticated"
  }
}
```

JSONL is useful because events can be processed individually without parsing one large JSON structure.

It is also suitable for processing with:

- Python;
- jq;
- Logstash;
- Fluent Bit;
- Filebeat;
- Splunk;
- Wazuh;
- Elastic Stack;
- custom SIEM collectors.

10. WordPress Activity Monitoring

WP bloggar records several categories of WordPress activity.

10.1 Authentication

Monitored authentication events include:

- successful login;
- failed login;
- logout;
- repeated authentication failures.

Authentication records can provide useful evidence during investigation of:

- credential attacks;
- password guessing;
- unauthorised access;
- compromised administrator accounts.

11. User Administration Monitoring

WP bloggar monitors important user-management actions including:

- user creation;
- user deletion;
- user profile changes;
- role changes;
- administrator privilege assignment.

Administrator-related events should be treated as particularly important.

For example:

Subscriber → Administrator

represents a significantly different security event from an ordinary profile update.

12. Plugin Monitoring

The system records important WordPress plugin lifecycle events including:

- plugin activation;
- plugin deactivation;
- plugin deletion;
- plugin upgrades.

Plugin activity is security relevant because compromised WordPress environments frequently use malicious or modified plugins for persistence or code execution.

13. Theme Monitoring

WP bloggar records relevant theme activity such as:

- theme changes;
- theme switching;

- theme upgrades.

Theme files may contain executable PHP and should therefore be included in filesystem integrity monitoring.

14. WordPress Update Monitoring

WP bloggar records upgrade activity involving:

- WordPress Core;
- plugins;
- themes.

This helps distinguish legitimate maintenance-related file changes from unexplained filesystem modifications.

15. Content Monitoring

The logger records selected content events including:

- post creation;
- page creation;
- post/page updates;
- deletion.

Content logging is primarily intended to provide administrative accountability rather than to record the full contents of every post.

16. Media Monitoring

WP bloggar monitors media-related activity including:

- media uploads;
- media deletion.

This is particularly relevant when combined with filesystem monitoring because the WordPress uploads directory should normally contain media and document assets rather than arbitrary executable server-side PHP code.

17. Configuration Monitoring

Changes to WordPress options can also be recorded.

Configuration changes may reveal modifications affecting:

- site behaviour;
- authentication;
- URLs;

- plugins;
- security controls;
- application operation.

Not every configuration change necessarily represents malicious activity. Events should therefore be interpreted in context.

18. Security Events

Version 1.1 introduces additional security-oriented monitoring.

The WordPress administration interface contains:

bloggar

├── Activity

└── Security Events

The Security Events interface provides a more focused view of events relevant to investigation and security monitoring.

19. Filesystem Integrity Monitoring

WP bloggar v1.1 provides filesystem integrity monitoring for selected WordPress areas.

Important monitored targets include:

wp-config.php

.htaccess

wp-admin/

wp-includes/

wp-content/plugins/

wp-content/themes/

wp-content/mu-plugins/

The scanner calculates cryptographic file hashes and compares the current filesystem state with a previously established baseline.

20. Integrity Baseline

The integrity state is stored in:

/var/log/wp-bloggar/integrity-state.json

The baseline represents the expected state of monitored files.

A subsequent scan can identify:

file_added

file_modified

file_deleted

21. Baseline Security Requirement

The initial baseline should be generated only when the administrator has reasonable confidence that the WordPress installation is clean.

This is important.

If a malicious file already exists when the baseline is generated, that file may become part of the trusted baseline.

Recommended sequence:

Incident investigation

↓

Clean WordPress Core

↓

Verify plugins

↓

Verify themes

↓

Inspect uploads

↓

Check wp-config.php

↓

Check .htaccess

↓

Create WP bloggar baseline

The baseline is therefore an integrity reference, not a malware scanner.

22. SHA-256 Integrity Hashing

WP bloggar uses SHA-256 hashes to identify changes to monitored files.

Conceptually:

File

↓

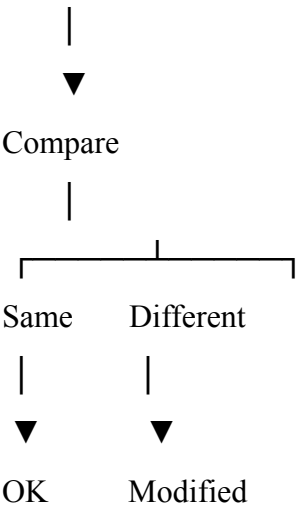
SHA-256

↓

Stored Baseline Hash

During later scans:

Current SHA-256



A changed hash indicates that the file contents have changed.

It does not by itself determine whether the modification was malicious.

23. PHP in Uploads Detection

A significant security check introduced in v1.1 is executable-file detection within the WordPress uploads area.

Typical WordPress uploads contain files such as:

.jpg

.jpeg

.png

.webp

.gif

.pdf

.docx

.xlsx

Unexpected PHP files may require investigation.

Examples:

/wp-content/uploads/2026/08/image.php

/wp-content/uploads/cache/ws.php

The presence of PHP does not automatically prove compromise, but it should normally be investigated.

24. Suspicious Filename Detection

The scanner can flag filenames commonly associated with malicious scripts or web shells.

Examples include names such as:

shell.php

ws.php

wso.php

c99.php

r57.php

about.php

Filename detection is an indicator only.

A legitimate file could have an unusual name, while sophisticated malware can use completely normal-looking filenames.

Security investigation should therefore include file contents, timestamps, ownership, hashes and surrounding activity.

25. Critical File Monitoring

Special attention should be given to:

wp-config.php

.htaccess

Changes to `wp-config.php` can affect:

- database connectivity;
- authentication secrets;
- WordPress constants;
- debugging;
- security settings;
- application behaviour.

Changes to `.htaccess` can affect:

- URL rewriting;
- access control;
- redirects;
- PHP execution;
- malicious traffic forwarding.

Unexpected modifications should be investigated promptly.

26. WordPress Core Integrity

The scanner monitors:

/wp-admin/

/wp-includes/

These directories contain WordPress Core components.

Under normal operation, unexpected PHP files or modifications within Core directories should be treated seriously unless they correspond to a legitimate WordPress upgrade.

For stronger verification, suspicious Core files should also be compared against the official WordPress release package for the exact installed version.

27. Plugin and Theme Integrity

The scanner monitors plugin and theme files for:

- creation;
- modification;
- deletion.

This can identify changes that bypass normal WordPress administrative actions.

For example:

Attacker gains arbitrary file-write capability

↓

Writes malicious PHP into plugin directory

↓

No normal "Plugin Activated" hook occurs

↓

Integrity scanner detects changed/new file

This addresses an important limitation of WordPress-hook-only auditing.

28. MU-Plugin Integrity

The MU-plugin directory is itself monitored.

This is important because security software should also monitor its own execution environment.

Changes involving:

/wp-content/mu-plugins/

should therefore receive careful review.

However, WP bloggar should not be considered tamper-proof against an attacker who obtains root or equivalent filesystem control.

29. Brute-Force Indicators

Version 1.1 tracks repeated authentication failures.

Security thresholds include:

5 failed attempts

10 failed attempts

20 failed attempts

These thresholds provide escalation indicators for repeated login failures from the same source.

They are detection events rather than automatic blocking rules.

Blocking should generally be implemented separately through controls such as:

- firewall rules;
- Fail2ban;
- reverse proxy controls;
- WAF;
- dedicated authentication protection.

30. XML-RPC Activity

WP bloggar v1.1 records relevant XML-RPC activity.

WordPress XML-RPC has legitimate use cases but can also appear in:

- authentication attacks;
- automated probing;
- legacy integration abuse.

If XML-RPC is not required by the site, administrators may independently consider disabling or restricting it.

Logging does not itself disable XML-RPC.

31. REST API User Activity

Security-sensitive REST activity involving users can also be logged.

This helps identify administrative actions that may occur through APIs rather than through conventional WordPress administration screens.

32. Automatic Security Scanning

WP bloggar uses WP-Cron to schedule periodic integrity checks.

The intended interval in v1.1 is approximately:

15 minutes

However, WordPress WP-Cron is traffic-driven.

Therefore:

15 minutes ≠ guaranteed execution every 15 minutes

On a low-traffic site, scans may occur later.

For security-critical deployments, a real Linux cron or system-level monitoring mechanism should be considered in a future implementation.

33. Manual Security Scan

Administrators can manually initiate a security scan from:

WP Admin

↓

bloggar

↓

Security Events

↓

Run Security Scan Now

This is useful:

- after WordPress upgrades;
- after plugin updates;
- after incident remediation;
- after restoring a backup;
- during investigation;
- when suspicious activity is observed.

34. WordPress Administration Interface

WP bloggar provides a web-based viewer within WordPress administration.

Main navigation:

bloggar

├── Activity

└── Security Events

The interface is intended for viewing logs and security events.

It is not necessary to directly browse to internal files such as:

/wp-content/wp-bloggar/admin/admin.php

Internal PHP components should not be directly exposed as standalone web applications.

35. Activity Viewer

The Activity interface provides access to recent records.

Typical information includes:

Field	Purpose
Timestamp	UTC event time
Severity	Security importance
Event	Event identifier
User	Associated WordPress user
IP	Source address
Details	Event-specific context

The interface provides a convenient operational view while the JSONL file remains the underlying audit source.

36. Severity Model

Events may be classified into levels such as:

INFO

MEDIUM

HIGH

CRITICAL

Conceptually:

INFO

Routine administrative activity.

Examples:

Successful login

Content update

Logout

MEDIUM

Activity requiring greater awareness.

Examples:

Failed login

Media upload

Profile modification

HIGH

Security-sensitive administrative activity.

Examples:

Plugin activation

Plugin deletion

Theme change

User deletion

CRITICAL

Activity with significant security implications.

Examples:

Administrator created

Administrator privilege assigned

Critical integrity violation

Suspicious executable file discovered

Severity should assist triage rather than substitute for investigation.

37. Viewing Logs Through SSH

List available logs:

sudo ls -lah /var/log/wp-bloggar/

View the current month's log:

sudo cat /var/log/wp-bloggar/wp-bloggar-2026-08.jsonl

View the last 50 events:

sudo tail -n 50 /var/log/wp-bloggar/wp-bloggar-2026-08.jsonl

Monitor new events in real time:

sudo tail -f /var/log/wp-bloggar/wp-bloggar-2026-08.jsonl

Press:

Ctrl+C

to stop `tail -f`.

38. Searching Logs

Search for failed logins:

grep "login_failed" /var/log/wp-bloggar/wp-bloggar-2026-08.jsonl

Search for critical events:

grep -i "critical" /var/log/wp-bloggar/wp-bloggar-2026-08.jsonl

Search for a particular IP address:

grep "192.0.2.10" /var/log/wp-bloggar/wp-bloggar-2026-08.jsonl

Search for file modifications:

grep "file_modified" /var/log/wp-bloggar/wp-bloggar-2026-08.jsonl

39. Processing Logs with jq

If `jq` is installed:

cat /var/log/wp-bloggar/wp-bloggar-2026-08.jsonl | jq .

Filter critical records:

jq 'select(.severity == "critical")' \

/var/log/wp-bloggar/wp-bloggar-2026-08.jsonl

This structured format makes WP bloggar suitable for automated security analytics.

40. Recommended File Ownership

The application code should ideally not be writable by the Apache/PHP process during normal production operation.

Conceptually:

Plugin Code

Owner: root

Writable by web server: NO

Log Directory

Owner/Group configured for controlled logging

Writable by logger: YES

Example deployment permissions must be adapted to the server environment.

Avoid:

chmod -R 777

on WordPress directories.

41. Protecting the Logger

A security logger is valuable only if its records and code are appropriately protected.

Recommended controls include:

- restrict write access to application code;
- keep logs outside the web root;
- restrict SSH access;
- use SSH keys;
- protect administrator accounts;
- use least privilege;
- maintain offline backups;
- monitor changes to MU plugins;
- forward important logs to another host or SIEM where possible.

A local logger cannot provide complete protection against an attacker with root-level server access.

42. Sensitive Data

Security logging should avoid collecting unnecessary secrets.

WP bloggar should never intentionally record:

- passwords;
- authentication cookies;
- session tokens;
- API secrets;
- private keys;
- WordPress nonces where unnecessary;
- database passwords;
- full sensitive POST bodies.

The objective is security visibility without unnecessarily increasing sensitive-data exposure.

43. Operational Testing

After installation, test the logger.

Test 1 — Login

Open SSH:

sudo tail -f /var/log/wp-bloggar/wp-bloggar-2026-08.jsonl

Log out and log back into WordPress.

Confirm a login event appears.

Test 2 — Failed Authentication

Attempt one controlled incorrect login.

Verify the failure is recorded.

Do not conduct uncontrolled brute-force testing against a production site.

Test 3 — Content Modification

Edit a test page and save it.

Confirm the update appears in the activity log.

Test 4 — Security Scan

Navigate to:

bloggar → Security Events

Run:

Run Security Scan Now

Verify that the integrity scan completes.

44. Interpreting Integrity Alerts

An integrity alert means:

The current filesystem differs from the stored baseline.

It does **not** automatically mean:

> The server has been compromised.

Legitimate causes include:

- WordPress updates;
- plugin updates;
- theme updates;
- administrator maintenance;
- configuration changes.

Potentially malicious causes include:

- web-shell upload;
- compromised administrator;
- vulnerable plugin exploitation;
- arbitrary file-write vulnerability;
- stolen SSH credentials;
- compromised hosting account.

Investigation is required to determine the cause.

45. Incident Response Use

When suspicious activity is detected, preserve evidence before unnecessarily changing files.

Useful evidence includes:

WP bloggar JSONL logs

Apache access logs

Apache error logs

PHP logs

Authentication logs

WordPress files

Database

File timestamps

File ownership

Cryptographic hashes

WP bloggar should be considered one evidence source within a broader investigation.

46. Known Limitations

WP bloggar has several important limitations.

WordPress dependency

Most activity monitoring depends on WordPress executing successfully.

WP-Cron timing

Scheduled integrity scans are not guaranteed to execute at exact intervals.

Local log compromise

An attacker with sufficient filesystem privileges may be able to modify local logs.

Baseline trust

An integrity baseline created after compromise may incorporate malicious files.

Signature detection

Suspicious filename detection cannot identify every web shell.

No automatic malware classification

A modified file is not automatically classified as malicious.

No automatic blocking

Logging and detection do not automatically block attackers.

47. Recommended Production Security Stack

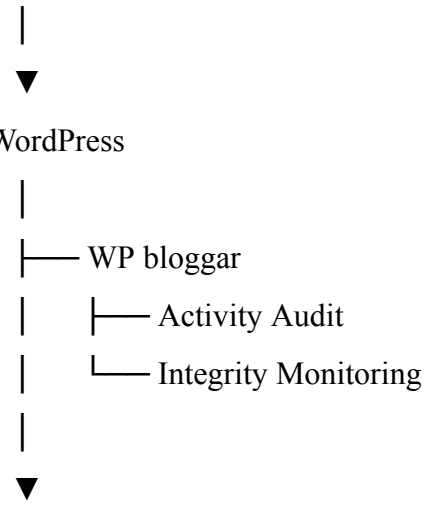
WP bloggar is best used as one layer:

Internet

Firewall / WAF

Apache

Hardened PHP



Additional layers should include:

- OS patching;
- WordPress Core updates;
- plugin/theme updates;
- MFA where appropriate;
- strong authentication;
- least privilege;
- SSH key authentication;
- firewall controls;
- backups;
- external monitoring.

48. Future Development

Potential future versions could introduce:

v1.2

- configurable event selection;
- configurable scan interval;
- log retention management;
- administrator filters;
- CSV/JSON export;
- trusted-change baseline update workflow;
- exclusion lists.

v1.3

- email security alerts;
- webhook integration;
- remote syslog;

- SIEM forwarding;
- Wazuh integration;
- event correlation;
- administrator login anomaly detection.

v2.0

- independent system cron scanner;
- external immutable logging;
- malware indicators;
- WordPress Core checksum verification;
- plugin integrity intelligence;
- security dashboard;
- risk scoring;
- incident timeline generation.

49. SIEM Integration

Because WP bloggar uses JSONL, future integrations can ingest:

/var/log/wp-bloggar/*.jsonl

Potential platforms include:

- Wazuh;
- Splunk;
- Elastic Stack;
- Graylog;
- Microsoft Sentinel through an appropriate collection pipeline;
- custom SOC platforms.

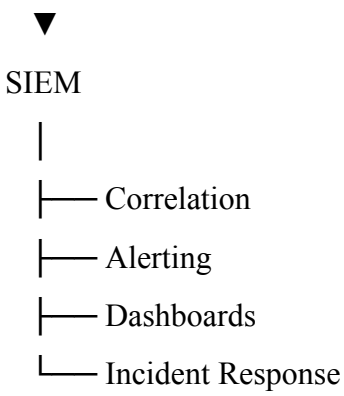
A future architecture could use:

WordPress

WP bloggar

Local JSONL

Log Forwarder



Remote logging is particularly valuable because it makes audit evidence harder for an attacker who controls the WordPress server to erase.

50. Security Philosophy

WP bloggar follows a simple principle:

Record important activity locally, detect unexpected filesystem changes, and preserve sufficient context for investigation.

The system deliberately separates:

Activity Logging

+

Filesystem Integrity

+

Security Event Triage

This provides better visibility than relying solely on standard WordPress administrative activity.

51. Quick Reference

Item	Configuration
Name	WP bloggar
Version	1.1.0

Deployment	Must-Use Plugin
MU Bootstrap	`/wp-content/mu-plugins/wp-bloggar.php`
Application	`/wp-content/wp-bloggar/`
Primary Logs	`/var/log/wp-bloggar/`
Fallback Logs	`/wp-content/uploads/wp-bloggar/`
Format	JSONL
Rotation	Monthly
Integrity Algorithm	SHA-256
Baseline	`integrity-state.json`
Admin UI	WP Admin → bloggar
Security UI	bloggar → Security Events
Scheduled Scan	Approximately every 15 minutes via WP-Cron
Manual Scan	Supported

52. Quick SSH Commands

Check installation:

ls -lah /var/www/html/wp/wp-content/mu-plugins/

ls -lah /var/www/html/wp/wp-content/wp-bloggar/

Check logs:

ls -lah /var/log/wp-bloggar/

Follow current log:

tail -f /var/log/wp-bloggar/wp-bloggar-$(date +%Y-%m).jsonl

Check integrity state:

ls -lh /var/log/wp-bloggar/integrity-state.json

Validate MU bootstrap PHP:

php -l /var/www/html/wp/wp-content/mu-plugins/wp-bloggar.php

53. Licence

A licence should be explicitly selected before public distribution.

For an open-source WordPress project, a GPL-compatible licence should be considered.

Add the selected licence as:

LICENSE

in the repository root.

54. Disclaimer

WP bloggar is a security monitoring and audit-support tool.

It does not guarantee detection or prevention of compromise. Security events and integrity alerts require appropriate technical interpretation.

Always maintain independent backups and use layered server, application and network security controls.

WP bloggar v1.1.0

WordPress Activity Logging • Security Event Monitoring • Filesystem Integrity